ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY

AUTONOMOUS INSTITUTION

Approved by AICTE & Affiliated to Anna University

NBA Accredited for BE (ECE, EEE, MECH) | Accredited by NAAC with A+ Grade

Anjugramam - Kanyakumari Main Road, Palkulam,Variyoor P.O. - 629 401, Kanyakumari District.

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

NAME OF THE STUDENT:

REG NUMBER:

YEAR:

COURSE CODE : 24EE404

COURSE NAME: IOT SENSORS AND DEVICES

LIST OF EXPERIMENTS

EX.NO	DATE	EXPERIMENT	PAGE NO.	MARKS	SIGNATURE
1		Building basic circuit diagrams using breadboard and Working of a Multimeter.			
2		Simple circuit using IC on breadboard.			
3		Simple Relay circuit design for ON-OFF condition.			
4		Switch on an LED if a button is pressed.			
5		Changing brightness of LED using potentiometer.			
6		Change the brightness of LED (Fade in/ Fade out) using PWM.			
7		DC motor speed control using serial communication.			
8		Interfacing Wi-Fi module with Arduino.			
9		Design a simple circuit to measure the pH value of wastewater.			
10		Design a simple circuit to apply Hall effect sensor			
11		Sending information about the patient in home to the doctor's PC/mobile.			
12		Develop an IoT System for real time applications.			

EX.NO:	1	BUILDING BASIC CIRCUIT DIAGRAMS USING BREADBOARD AND WORKING OF A MULTIMETER
DATE:		

Aim:

To measure the voltage and current through the LED using the multimeter

Components Required:

S. No	Apparatus	Quantity
01	LED	1
02	Resistance (10K)	1
03	Battery (9V)	1
04	Bread board and connecting wires	

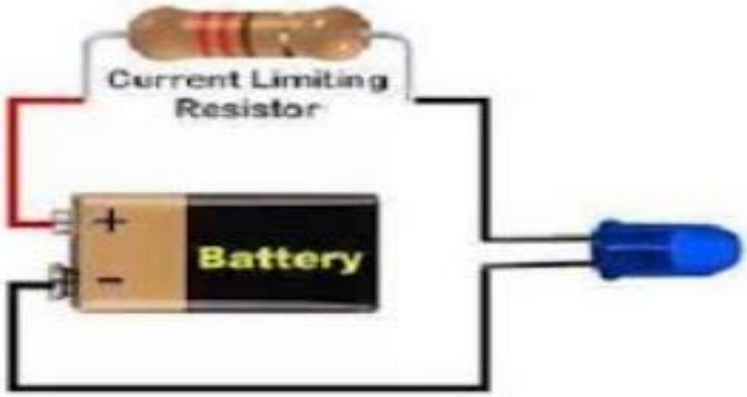
Theory:

When a resistor is connected in series with an LED in a circuit to protect the light-emitting diode by restricting the flow of current through them is known as an LED resistor. In a LED circuit, a small change within voltage supply can cause the LED to glow very brightly or dim or they can even damage also. So a series resistor is used to avoid the flow of current effect through LED slightly.

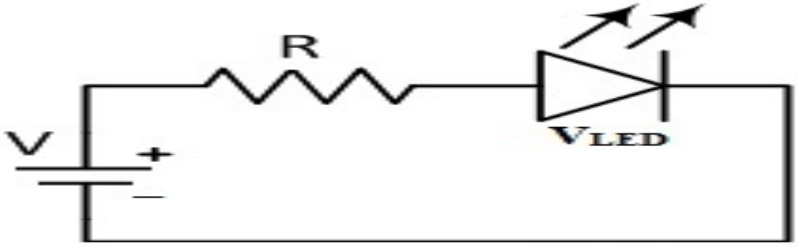
An LED has two terminals a positive & a negative where the positive is known as the anode and the negative is known as the cathode. The LED schematic symbol is similar to the diode apart from two arrows outwards. The anode is marked with a triangle & the cathode is marked with a line. Generally, the two terminals of an LED include one longer terminal and one shorter terminal where the longer term is the anode and the shorter terminal is the cathode. Resistors have no polarity so they can be arranged in any way so they work fine in any direction.

The polarity of batteries can be determined very easily because the two terminals of the battery the positive & negative will be marked through a positive (‘+’) and a negative (‘-‘) symbol. The connection of this circuit can be done as per the simple circuit diagram shown below. Here, the resistor is connected in series with a battery and voltage source.

Model:



Circuit diagram:



Tabulation:

1	**Voltage in battery**	
2	**Voltage in LED**	
3	**Current in LED**	

Result:

Thus, the voltage and current through the LED are measured by the multimeter

EX.NO:	2	SIMPLE CIRCUIT USING IC ON BREADBOARD
DATE:		

Aim:

To design and build an LED blink circuit using a 555 timer IC

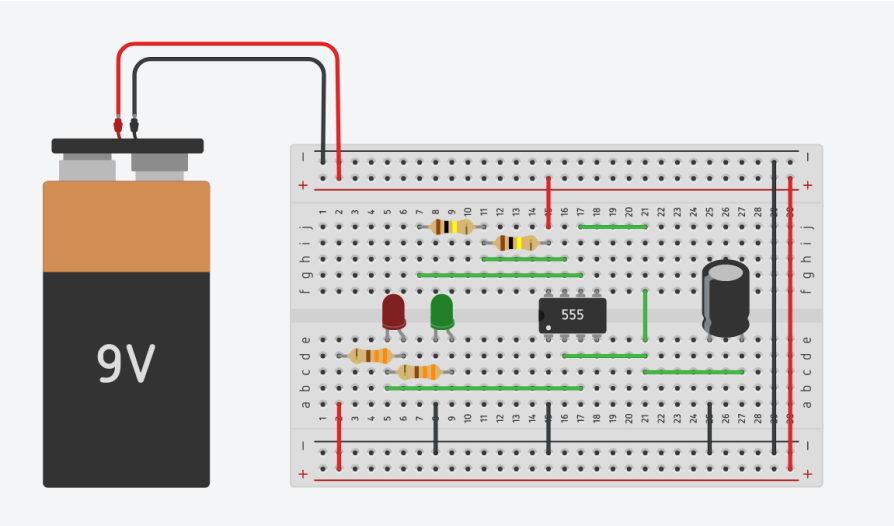
Components Required:

S. No	Apparatus	Quantity
01	LED	2
02	Resistance (330 Ω) and (100k Ω)	2 , 2
03	Battery (9V)	1
04	Capacitor (110uF)	1
05	555 Timer IC	1
06	Bread board and connecting wires	

Theory:

The 555 timer IC in astable mode generates a square wave, causing an LED to blink. The frequency of blinking depends on the external resistors (R1, R2) and capacitor (C) connected to the timer. As the capacitor charges and discharges, the output alternates between HIGH (LED ON) and LOW (LED OFF). The timing cycle is determined by the resistor and capacitor values, with formulas to calculate high time, low time, and frequency. By adjusting these components, the LED's blinking speed can be controlled. This simple circuit is widely used for timing, blinking, and pulse generation applications.

Circuit Diagram:



Result:

The LED was successfully blinked using IC555 timer.

EX.NO:	3	SIMPLE RELAY CIRCUIT DESIGN FOR ON-OFF CONDITION
DATE:		

Aim:

To design a basic relay circuit that can control a LED using a simple ON-OFF condition

Components Required:

Arduino UNO
Breadboard
Relay
LED
Connecting Wires
USB Cable

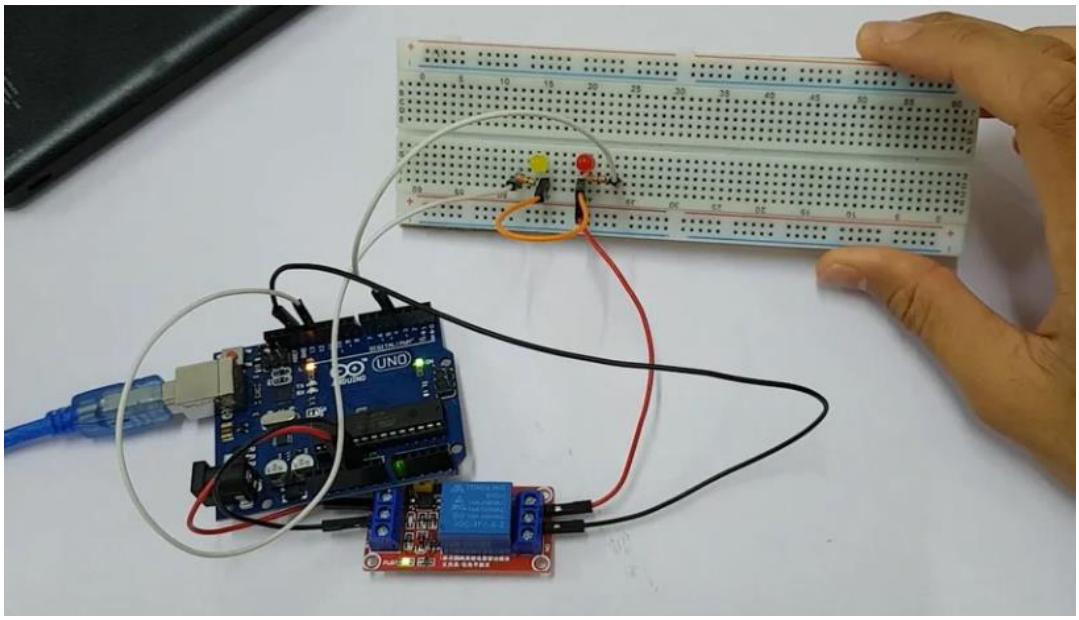
Platform:

Arduino IDE

Program:

```
#define LEDgreen 4 //Connect Digital Pin 4 on Arduino to Green LED (+ 330 ohm resistor)
 and then to "NO" terminal on relay module
#define LEDyellow 12 //Connect Digital Pin 12 on Arduino to Yellow LED (+ 330 ohm
 resistor) and then to "NC" terminal on relay module

void setup() {
 //Setup all the Arduino Pins

 pinMode(LEDgreen, OUTPUT);
 pinMode(LEDyellow, OUTPUT);

 //Provide power to both LEDs
 digitalWrite(LEDgreen, HIGH);
 digitalWrite(LEDyellow, HIGH);

}
void loop() {
 digitalWrite(4, HIGH);  //Green LED on, Yellow LED off
 delay(1000);
 digitalWrite(4, LOW);   //Yellow LED on, Green LED off
 delay(1000);
 digitalWrite(12, HIGH);  //Relay 3 switches to NO
 delay(1000);
 digitalWrite(12, LOW);   //Relay 3 switches to NC
 delay(1000);
}
```

Output:



Result:

The Relay circuit successfully controls the LED ON-OFF condition.

EX.NO:	4	SWITCH ON AN LED IF A BUTTON IS PRESSED
DATE:		

Aim:

To design a simple circuit that switches on an LED when a button is pressed

Components Required:

Breadboard (generic)	-	1
Resistor 1k ohm	-	2
Jumper wires (generic)	-	5
Arduino UNO	-	1
LED (generic)	-	1
USB Cable	-	1

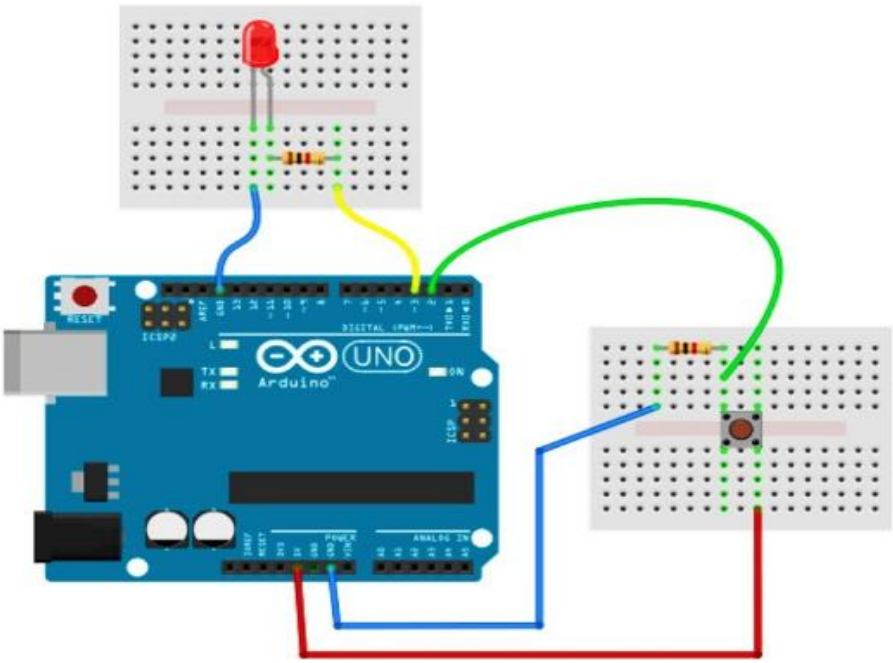
Platform:

Arduino IDE

Program:

```
const int BUTTON = 2;
const int LED = 3;
int BUTTONstate = 0;

void setup()
{
  pinMode(BUTTON, INPUT);
  pinMode(LED, OUTPUT);
}

void  loop()
{
  BUTTONstate = digitalRead(BUTTON);
  if (BUTTONstate == HIGH)
  {
    digitalWrite(LED, HIGH);
  }
  else{
    digitalWrite(LED,  LOW);
  }
}
```

Output:



Result:

Thus, the LED turned on when the button was pressed.

EX.NO:	5	CHANGING BRIGHTNESS OF LED USING POTENTIOMETER
DATE:		

Aim:

To design a circuit that controls the brightness of an LED using a potentiometer.

Components Required:

- Arduino Uno board.
- Breadboard.
- LED – any color.
- 220 Ohm resistor.
- Potentiometer.
- A bunch of wires (male to female).

Platform:

Arduino IDE

Theory:

Steps to build the circuit:

- As a best practice we'll start with the ground (GND). It's very important to make a common ground for all components. To do this we will first plug a black wire (black is convention for GND) between a GND pin of the Arduino and the "minus" line on the breadboard. From this "minus" line, then we will be able to connect all other grounds, which will make things easier to manage.

Add the LED:

- Connect the shorter leg of the LED to the ground. You can directly plug this leg into the "minus" line of the breadboard, or add a small black wire.
- Plug the other (longer) leg of the LED to an independent line on the breadboard.
- From this leg, add a 220 Ohm resistor to yet another line.
- Add a wire between the other side of the resistor and a PWM-compatible digital pin (so we can control the brightness). Here on Arduino Uno you can choose between pins 3, 5, 6, 9, 10, and 11 – you can recognize PWM compatibility with the "~" next to the pin number.

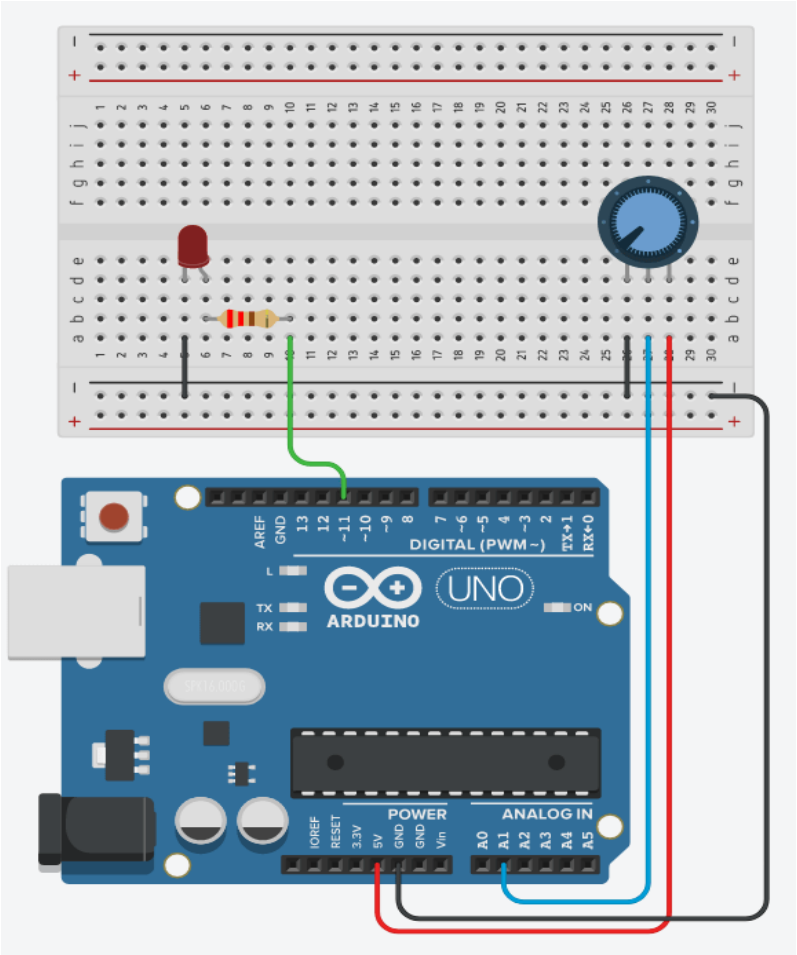
Add the Potentiometer:

- Plug the 3 legs of the potentiometer to 3 different lines on the breadboard.
- Connect the extreme left (or right) leg to GND.
- Connect the other extreme leg to 5V on the Arduino.
- Add a wire between the middle pin and an analog pin.

Program:

```
#define LED_PIN 11
#define POTENTIOMETER_PIN A1
void setup()
{
  pinMode(LED_PIN, OUTPUT);
}
void loop()
{
  int potentiometerValue = analogRead(POTENTIOMETER_PIN);
  int brightness = potentiometerValue / 4;
  analogWrite(LED_PIN, brightness);
}
```

OUTPUT



Result:

Thus, the brightness of an LED was varied by using a potentiometer.

EX.NO:	6	CHANGE THE BRIGHTNESS OF LED (FADE IN/ FADE OUT) USING PWM
DATE:		

Aim:

To design a circuit that controls the brightness of an LED using Pulse Width Modulation (PWM)

Components Required:

Arduino Uno	-	1
Breadboard	-	1
LED	-	1
Jumper wire (male-male)	-	2
470 Ohm (gold-brown-violet-yellow) resistor	-	1

Platform:

Arduino IDE

Theory:

Make the LED smoothly turn on and off (so-called fading). For this we are going to use a technique called PWM (Pulse Width Modulation).
Not all pins on the Arduino support PWM. This only applies to the Pins marked with a ~.

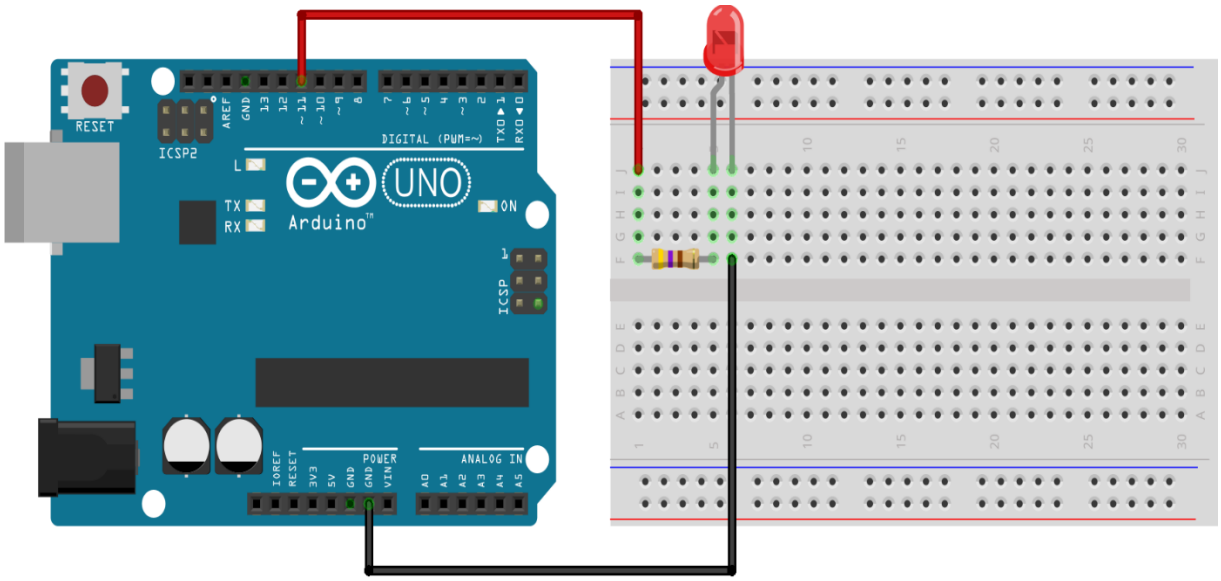
Fig: Pin 11, 10 and 9 support PWM (Pulse Width Modulation)

Program:

```
int LED Pin = 11; // PWM capable pin

void setup() {
pinMode(LED Pin, OUTPUT);
}

void loop() {
  analogWrite(LED Pin, 128); // Run motor at 50% speed (0-255)
  delay(2000); // Run for 2 seconds
  analogWrite(LED Pin, 57);
  delay(2000); // Run for 2 seconds
  analogWrite(LED Pin, 0);   // Stop motor
  delay(2000); // Wait for 2 seconds
}
```

Output:

In this circuit we see that the anode of the LED is connected via a resistor to pin 11 on the Arduino. Then the cathode of the LED is connected to the ground (GND) on the Arduino



Result:

Thus, the brightness of the LED will gradually decrease using PWM.

EX.NO:	7	DC MOTOR SPEED CONTROL USING SERIAL COMMUNICATION
DATE:		

Aim:

To design a circuit that controls the speed of Motor using serial communication.

Components Required:

Arduino Uno	-	1
Breadboard	-	1
DC Motor	-	1
Jumper wire (male-male)	-	2

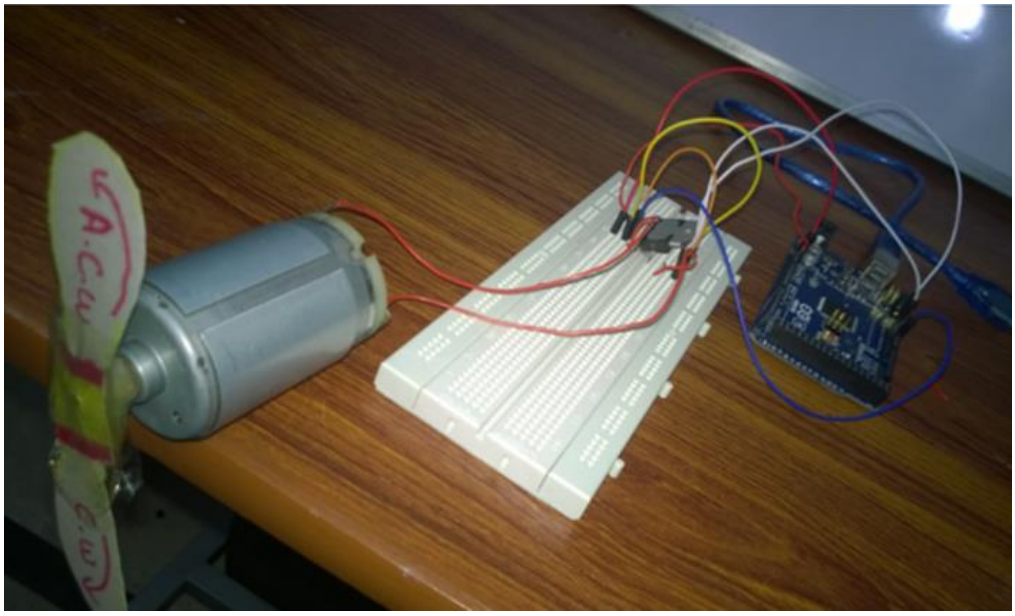
Platform:

Arduino IDE

Program:

```
int motorPin = 9; // PWM capable pin

void setup() {
pinMode(motorPin, OUTPUT);
}

void loop() {
  analogWrite(motorPin, 128); // Run motor at 50% speed (0-255)
  delay(2000); // Run for 2 seconds
  analogWrite(motorPin, 57);
  delay(2000); // Run for 2 seconds
  analogWrite(motorPin, 0);   // Stop motor
  delay(2000); // Wait for 2 seconds
}
```

OUTPUT



Result:

Thus, the speed of Dc Motor was controlled by PWM.

EX.NO:	8	INTERFACING WI-FI MODULE WITH ARDUINO
DATE:		

Aim:

To create a simple web server that can control an LED via a web browser.

Apparatus Required:

- **ESP32 board**
- **LED and a 220-ohm resistor**
- **Jumper wires**
- **Breadboard**

Theory:

1. ESP32 Overview:
The **ESP32** is a powerful microcontroller with built-in **Wi-Fi** and **Bluetooth** capabilities. It is widely used for IoT (Internet of Things) applications, thanks to its ability to connect to the internet and interact with external devices (like sensors, motors, LEDs, etc.).

2. Wi-Fi Connection:
The ESP32 has an integrated **Wi-Fi module** that allows it to connect to wireless networks. In the program, the ESP32 connects to your local Wi-Fi network by specifying an **SSID** (network name) and **password**. Once connected, the ESP32 gets an **IP address** that allows it to communicate over the network.

- **WiFi.begin(ssid, password):** This function is used to initiate the connection to the Wi-Fi network.
- **WiFi.status():** Checks whether the ESP32 has successfully connected to the Wi-Fi network. It returns WL_CONNECTED once the connection is established.

3. Web Server:
Once the ESP32 is connected to Wi-Fi, it can act as a **web server**, which is a service that serves content (like HTML pages) to clients (like web browsers). In this case, the ESP32 serves a web page that contains buttons to control an LED.

- **WiFiServer server(80):** Initializes a web server that listens for requests on **port 80**, the default port for HTTP traffic.
- **server.begin():** Starts the web server, making the ESP32 ready to accept incoming HTTP requests.

4. HTTP Requests and Responses:
When a user accesses the web page by entering the ESP32's **IP address** in a browser, the following happens:

- The browser sends an **HTTP GET request** to the server hosted on the ESP32.
- The ESP32 then processes this request and sends back an **HTTP response**, typically containing HTML content.
- **client.println("HTTP/1.1 200 OK")**: This line is part of the HTTP header, indicating a successful request and that the content being sent is an HTML page.

5. HTML Web Page:
The ESP32 serves a simple **HTML page** containing buttons that the user can click to control the LED:

- **<a href="/LED_ON">**: This link sends a request to the server to turn the LED on.

- **<a href="/LED_OFF">**: This link sends a request to the server to turn the LED off.

6. Processing User Inputs:

When the user clicks a button on the web page:

- The **GET request** is sent to the ESP32, and the program checks if the request is for turning the LED on or off.
- **currentLine.indexOf("GET /LED_ON")** checks if the request is to turn the LED on. If true, it sets the LED pin to **HIGH**, turning the LED on.
- **currentLine.indexOf("GET /LED_OFF")** checks if the request is to turn the LED off. If true, it sets the LED pin to **LOW**, turning the LED off.

7. LED Control:

The **LED** is controlled by setting the **GPIO pin** (in this case, **GPIO 2**) to either HIGH or LOW:

- **HIGH**: Turns the LED on.
- **LOW**: Turns the LED off.

8. Client Disconnect:

Once the ESP32 processes the request and sends the HTML page back to the browser, it disconnects the client:

- **client.stop()**: This closes the connection after the response has been sent.

Procedure:

- Wi-Fi Connection: ESP32 connects to your Wi-Fi network.
- Web Server: ESP32 starts a server and listens for incoming requests.
- HTML Page: The ESP32 serves a web page with buttons to control the LED.
- Button Click: The user clicks a button on the web page, sending an HTTP request to the ESP32.
- Process Request: The ESP32 processes the request and turns the LED on or off based on the button clicked.
- Response**:** The ESP32 sends a new HTML page with updated button states to the user's browser.

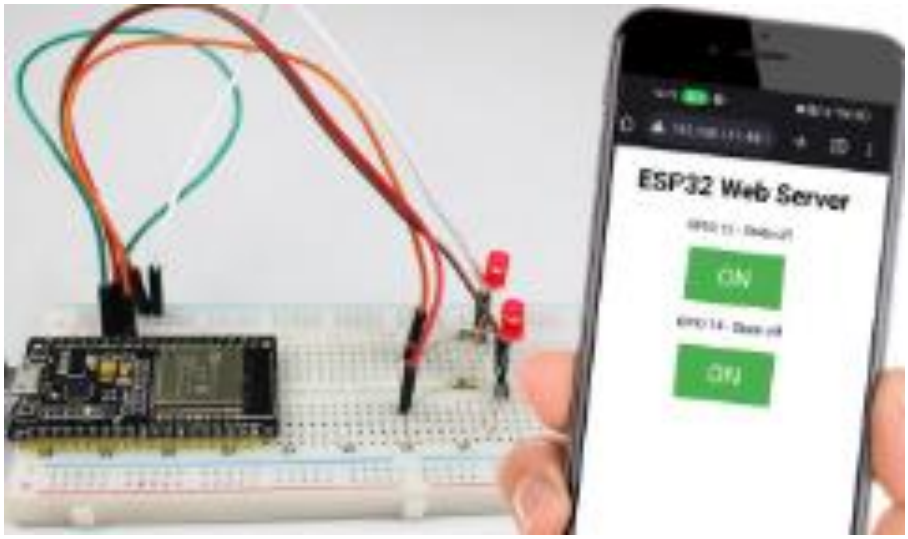
Program:

```
#include <WiFi.h>

// Replace with your network credentials
const char* ssid = "your_SSID";
const char* password = "your_PASSWORD";

// Set your LED GPIO pin
const int ledPin = 2;

WiFiServer server(80);

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);
```

```
  // Connect to Wi-Fi
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());

  server.begin();
}

void loop() {
  WiFiClient client = server.available();
  if (client) {
    Serial.println("New Client.");
    String currentLine = "";
    while (client.connected()) {
      if (client.available()) {
        char c = client.read();
        Serial.write(c);
        if (c == '\n') {
          if (currentLine.length() == 0) {
            // HTTP headers
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type:text/html");
            client.println();

            // Web Page HTML
            client.println("<!DOCTYPE html><html>");
            client.println("<head><title>ESP32 LED Control</title></head>");
            client.println("<body><h1>ESP32 Web Server</h1>");
            client.println("<p><a href=\"/LED_ON\"><button>Turn ON LED</button></a></p>");
```

```
            client.println("<p><a href=\"/LED_OFF\"><button>Turn OFF LED</button></a></p>");
            client.println("</body></html>");

            client.println();

            break;
          } else {
            // Check HTTP request
            if (currentLine.indexOf("GET /LED_ON") >= 0) {
              digitalWrite(ledPin, HIGH);
            }
            if (currentLine.indexOf("GET /LED_OFF") >= 0) {
              digitalWrite(ledPin, LOW);
            }
            currentLine = "";
          }
        } else if (c != '\r') {
          currentLine += c;
        }
      }
    }
    client.stop();
    Serial.println("Client Disconnected.");
  }
}
```

OUTPUT



Result:

Thus the LED was controlled via web browser.

EX.NO:	9	DESIGN A SIMPLE CIRCUIT TO MEASURE THE PH VALUE OF WASTEWATER
DATE:		

Aim:

The aim of this experiment is to design a simple circuit to measure the PH value of waste water.

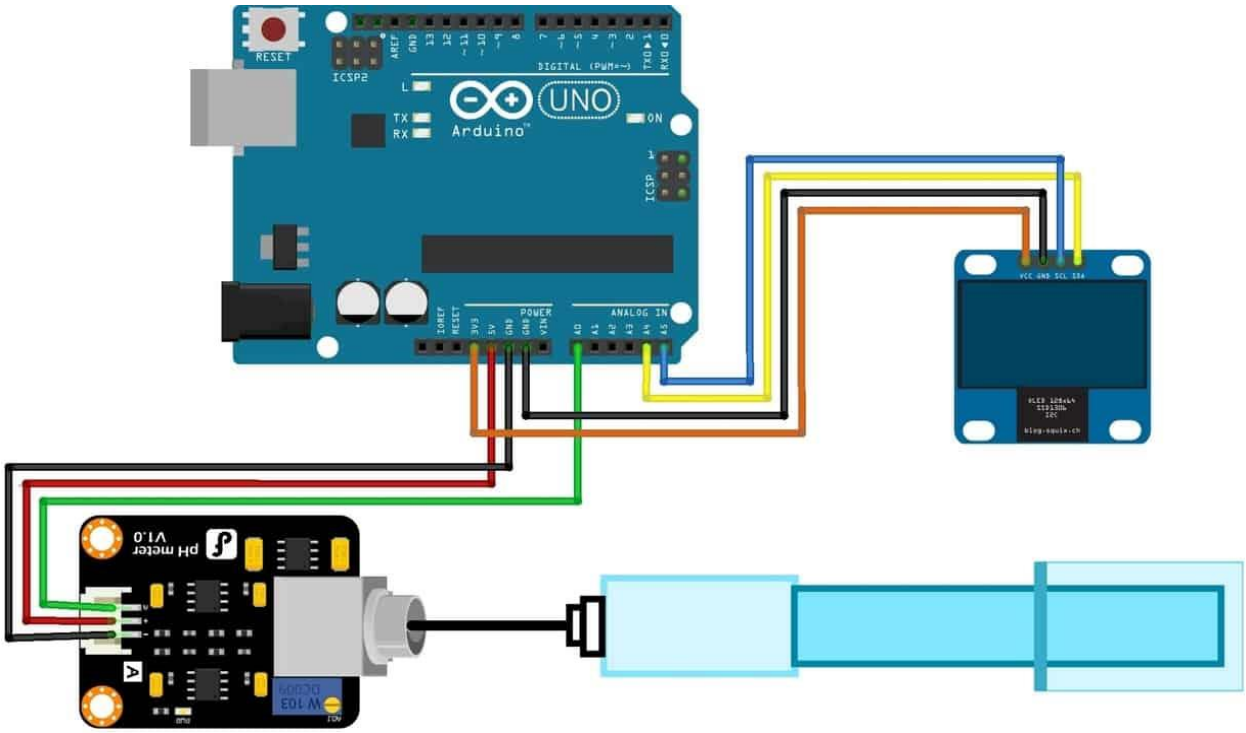
Platform:

Arduino IDE

Program:

```
int pHSensorPin = A0;  // Pin connected to the pH sensor
float voltage = 0;      // Voltage read from the sensor
float pHValue = 0;      // pH value of the solution

void setup() {
  Serial.begin(9600);  // Start serial communication
}

void loop() {
  voltage = analogRead(pHSensorPin) * (5.0 / 1023.0);  // Convert the analog reading to voltage
  pHValue = 3.5 * voltage + 0.5;  // Example formula, you'll need to calibrate this for accuracy

  // Print the voltage and pH value to the Serial Monitor
  Serial.print("Voltage: ");
  Serial.print (voltage);
  Serial.print("V, pH: ");
  Serial.println(pHValue);

  delay(1000);  // Wait for 1 second before taking the next reading
}
```

Block Diagram:



Result:

Thus the PH value of waste water was measured successfully

EX.NO:	10	DESIGN A SIMPLE CIRCUIT TO APPLY HALL EFFECT SENSOR
DATE:		

Aim:

The aim of this experiment is to design a simple circuit to apply Hall effect sensor

Components Required:

- Arduino Uno
- Jumper Wires
- Bread Board
- Hall effect sensor

Platform:

Arduino IDE

Theory:

Hall effect sensor is a digital sensor. It is use to detect the magnet. When you came the magnet near to Hall effect sensor, it outputs a HIGH (5V) voltage to its Vout pin. In this tutorial, we will learn how to read value of Hall effect sensor with Arduino. It is basically used to calculate the RPM of motor etc.

Pinout:

VCC –> VCC

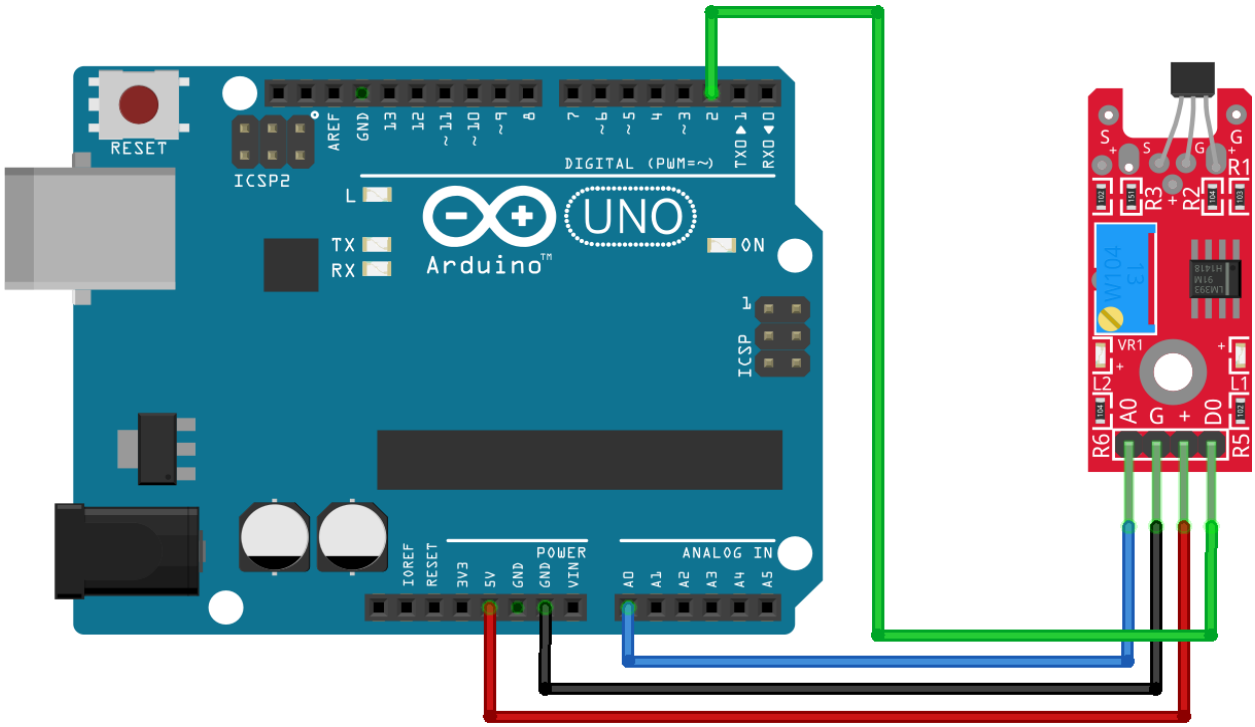
GND –> GND

DO –> D2

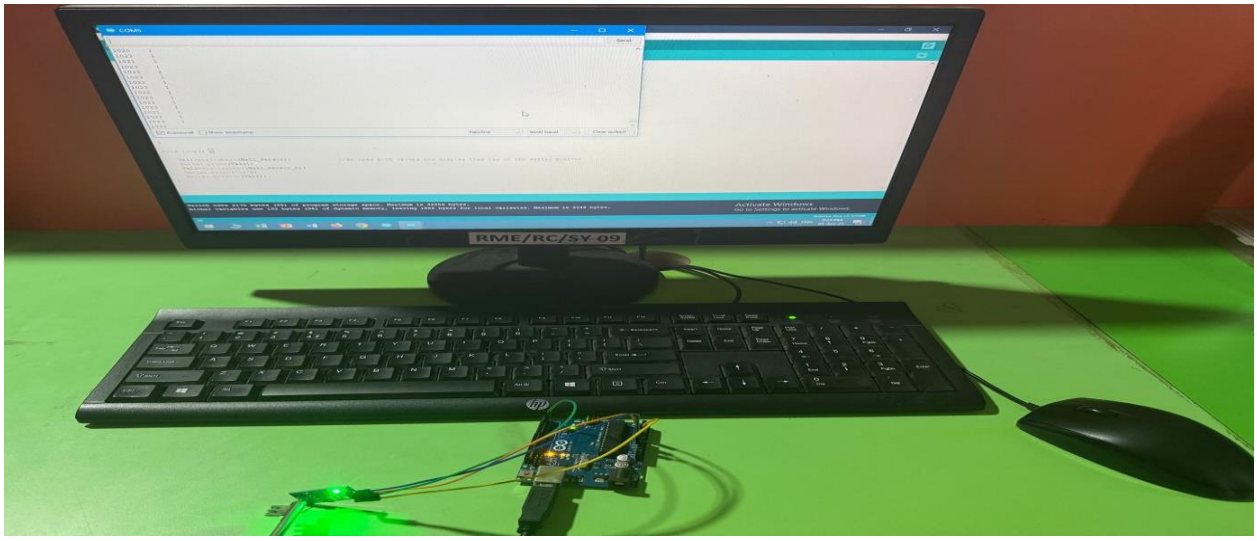
Program:

```
#define Hall_Sensor A0          //A0 used with analog output, D2 with digital output
#define Hall_Sensor_D 2

int Val1=0, Val2=0;             //Here you can store both values, the Val2 can be boolean


void setup() {
  Serial.begin(9600);
  pinMode(Hall_Sensor_D,INPUT);


}

void loop() {

   Val1=analogRead(Hall_Sensor);          //We read both values and display them raw on the serial monitor
   Serial.print(Val1);
   Val2=digitalRead(Hall_Sensor_D);
   Serial.print("\t");
   Serial.println(Val2);
}
```

Block Diagram:



OUTPUT:



Result:

Thus, the Hall effect sensor was done successfully

EX.NO:	11	SENDING INFORMATION ABOUT THE PATIENT IN HOME TO THE DOCTOR'S PC/MOBILE
DATE:		

Aim:

To send the information about the patient in home to the doctor's PC or mobile

Components :

1. **Arduino Board** (e.g., Arduino Uno, Nano, or any other suitable board)
2. **Sensors** for collecting patient data (e.g., heart rate sensor, temperature sensor, pulse oximeter)
3. **Wi-Fi or Bluetooth Module** (e.g., ESP8266, ESP32, or HC-05 Bluetooth module)
4. **Doctor's PC or Mobile Device**
5. **Server or Web Application** (optional for handling and storing the data securely)

Basic Setup:

1. **Sensor Integration**: Connect the sensors to the Arduino to collect the patient's data.
2. **Connectivity Module**: Use a Wi-Fi (ESP8266 or ESP32) or Bluetooth (HC-05) module to send the data from Arduino to the doctor's device.
3. **Sending Data**: The Arduino sends the collected data (e.g., heart rate, body temperature, etc.) to a server, database, or directly to a doctor's device.
4. **Receiving Data on Doctor's Device**: The doctor's PC or mobile device can retrieve the data through a custom app, web portal, or email.

Program

```
#include <ESP8266WiFi.h>
#include <DHT.h>
#include <ThingSpeak.h>

#define DHTPIN 2            // DHT11 data pin connected to GPIO2 (D4 on NodeMCU)
#define DHTTYPE DHT11       // Define the sensor type as DHT11
DHT dht(DHTPIN, DHTTYPE);   // Initialize DHT11 sensor

// Wi-Fi credentials
const char* ssid = "your_SSID";          // Replace with your Wi-Fi SSID
const char* password = "your_PASSWORD";  // Replace with your Wi-Fi password
```

```
// ThingSpeak channel settings
unsigned long channelID = YOUR_CHANNEL_ID;   // Replace with your ThingSpeak channel ID
const char* writeAPIKey = "YOUR_API_KEY";     // Replace with your ThingSpeak Write API Key

WiFiClient client;  // Create Wi-Fi client for ThingSpeak

void setup() {
  // Start serial communication for debugging
  Serial.begin(115200);

  // Connect to Wi-Fi
  WiFi.begin(ssid, password);
  Serial.println("Connecting to WiFi...");

  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting...");
  }
  Serial.println("Connected to WiFi!");

  // Initialize ThingSpeak
  ThingSpeak.begin(client);

  // Initialize DHT11 sensor
  dht.begin();
}

void loop() {
  // Read temperature and humidity values from the DHT11 sensor
  float temperature = dht.readTemperature(); // Temperature in Celsius
  float humidity = dht.readHumidity();       // Humidity in percentage

  // Check if reading failed
  if (isnan(temperature) || isnan(humidity)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }

  // Print the readings to the Serial Monitor (for debugging purposes)
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.print(" °C  Humidity: ");
  Serial.print(humidity);
  Serial.println(" %");

  // Send the data to ThingSpeak
  ThingSpeak.setField(1, temperature); // Field 1 for temperature
  ThingSpeak.setField(2, humidity);    // Field 2 for humidity
```

```
  // Write data to ThingSpeak channel
  int responseCode = ThingSpeak.writeFields(channelID, writeAPIKey);
  if (responseCode == 200) {
    Serial.println("Data successfully sent to ThingSpeak.");
  } else {
    Serial.print("Error sending data. Response code: ");
    Serial.println(responseCode);
  }

  // Wait for 20 seconds before sending data again (ThingSpeak update rate is 15 seconds minimum)
  delay(20000);
}
```

Result

Thus, the patient information was sent from home to a doctor’s PC or mobile

EX.NO:	**12**	**DEVELOP AN IOT SYSTEM FOR REAL TIME APPLICATIONS**
DATE:		

Aim:

To prepare an experiment on voice control home automation using Arduino.

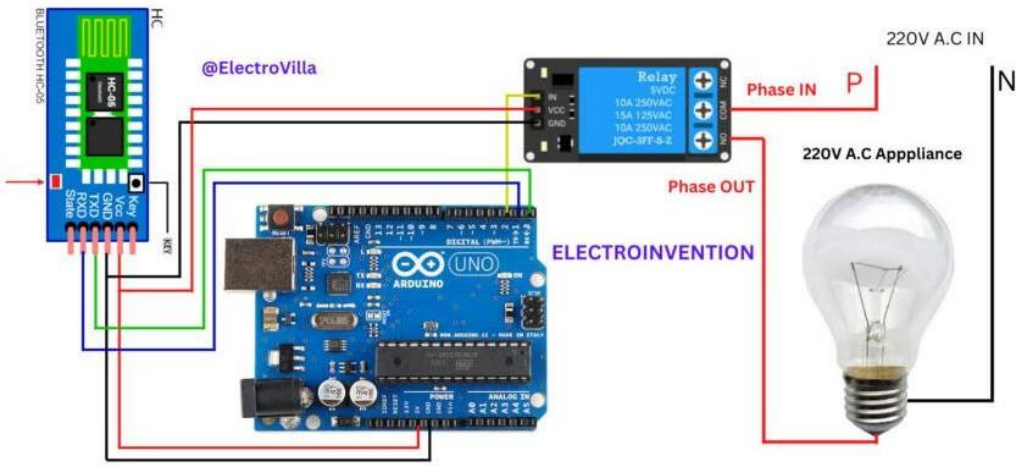
Components Required

- Arduino UNO
- HC-05 Bluetooth Module
- channel Relay Module(5v)
- holders
- Bulb
- 220v Electrical wire with a 2-pin male socket
- Sun board Piece
- Jumper wires
- Nuts & bolts
- And a Smartphone

Procedure:

- Using the above components we implement our system.
- An adapter of 12V output power supply is used as an input to the voice controlled arduino system.
- Relays are connected to the output pins of Arduino Uno, these are used as switches to the loads.
- For wireless communication system a Bluetooth module HC-05 is used as a remote which is connected to the control unit for sensing the signals sent by the android voice application.
- Then we need to launch the android based application – "ARDUINO BLUETOOTH CONTROLLER" on our smart phone.
- The microcontroller uses the port in serial mode. After reading the data it decodes the input value and sends a signal to the parallel port through which the relay circuit will be activated.

Circuit Diagram:

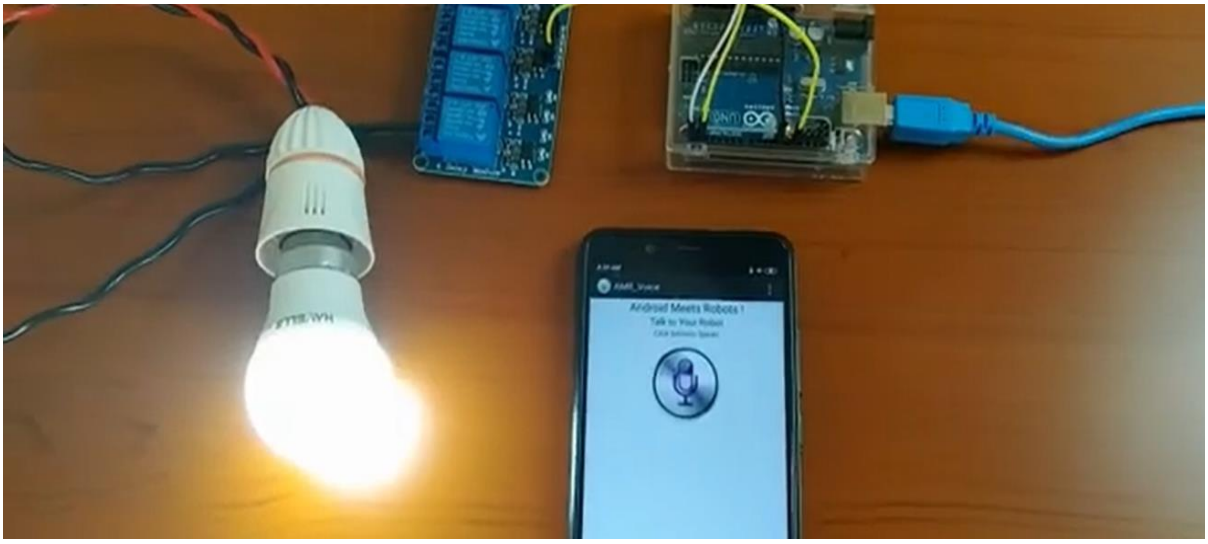


Program:

```
String voice;

#define relay 12

void setup()

{                                   // put your setup code here, to run once:

  Serial.begin(9600);

  pinMode(relay, OUTPUT);

  digitalWrite(relay, LOW);

}

  int relayVal;

void loop() {

  while (Serial.available())   //Check if there is an available byte to read

  {
```

```
  delay(10);                    //Delay added to make thing stable

  char c = Serial.read();       //Conduct a serial read

  if (c == '#') {break;}        //Exit the loop when the # is detected after the word

  voice += c;                   //Shorthand for voice = voice + c

  }

  if (voice.length() > 0) {

    Serial.println(voice);

  //-------Control Light------------//

       if(voice == "turn on the light")       // To tune ON the bulb

     {

     digitalWrite(relay, HIGH);

     }

  else if(voice == "turn off the light")      // To turn OFF the bulb

     {

    digitalWrite(relay, LOW);

     }

voice="";                           //Reset the variable after initiating

}

}
```

Output:



Result:

Thus the experiment for voice control home automation is done successfully.